



UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA

VICERRECTORADO ACADÉMICO

COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA

INGENIERÍA DE SOFTWARE II



ACTIVIDAD III PRODUCTO IV:

“Risk Storming Simulado para Fintech Mobile (Serverless + API IA)”

Docente:

Félix Marquez

Integrantes:

Gómez Andrés V-31.085.717

Ramírez Alexmary V-31.809.930

Silva José V-30.810.283

Vallenilla Daniel V-31.159.105

Actividad 3: Análisis Comparativo: Sesión Tradicional vs. Risk Storming

1. Tabla Comparativa de Enfoques

Criterio	Sesión Tradicional (PM Centrada)	Risk Storming (Multifuncional)
Liderazgo y Participación	Únicamente el Gerente de Proyectos (PM) con una lista de verificación (<i>checklist</i>).	Todo el equipo multifuncional (Desarrolladores, QA, DevOps, Arquitectos, PO).
Formato del Ejercicio	Entrevista/Llenado de hoja de cálculo en Word/Excel estático.	Sesión colaborativa, visual e interactiva sobre la arquitectura/flujo del sistema.
Perspectiva de Evaluación	Énfasis en riesgos de gestión, cronograma, presupuesto y alcance.	Énfasis en riesgos de arquitectura, ciberseguridad, dependencias y calidad del código.
Resultados y Integración	Registro de riesgos estático que se archiva y rara vez se actualiza.	Historias de Habilidad (<i>Enablers</i>) y <i>Spikes</i> integrados al <i>Product Backlog</i> .

2. ¿Por qué el Risk Storming es más efectivo para detectar riesgos técnicos y de ciberseguridad en la arquitectura?

- **Eliminación del Punto Ciego del PM:** El Gerente de Proyectos posee una visión de alto nivel orientada al negocio, pero carece de la visibilidad sobre el funcionamiento interno de la pila tecnológica. Vulnerabilidades como la inyección de dependencias maliciosas en paquetes de terceros (*Supply Chain Attacks*) o los problemas de modelo no determinista en llamadas a APIs de IA son detectables principalmente por quienes escriben y despliegan el código.

- **Análisis Situado en la Arquitectura (Mapa Visual):** El *Risk Storming* utiliza el diagrama de arquitectura real del software como lienzo. Al evaluar visualmente cómo interactúa cada microservicio o función *Serverless* con la base de datos o APIs externas, los desarrolladores y *DevOps* identifican fácilmente fallas de *Vendor Lock-in*, cuellos de botella y vectores de ataque que no aparecen en listas de verificación genéricas.
- **Perspectiva de Ciberseguridad Directa (DevSecOps):** La inclusión de *QA* e ingenieros de seguridad en el debate permite simular cómo un atacante explotaría una biblioteca obsoleta o un mal manejo de tokens de acceso, promoviendo el enfoque de *Security by Design* desde el inicio del desarrollo.
- **Apropiación Colectiva y Acción Inmediata:** Al participar activamente en la detección, el equipo técnico no percibe la gestión de riesgos como una carga burocrática del PM, sino como un trabajo técnico visible que se traduce inmediatamente en historias de usuario y *Spikes* dentro del *Sprint Backlog*.